EdgeShield: Autonomous Edge Caching & In-Memory DNS Gateway

Computer Networks Collaborative Systems Project Specification & Integration Briefing

Author: Pranjal Panda

Cross-platform implementation: Partner A on macOS (Apple Silicon), Partner B on Linux

1. Problem Statement & Mathematical Bottleneck Model

In institutional networks such as college campuses, end systems connect to a high-speed LAN (100 Mbps) that shares a rate-limited access link (for example 15 Mbps) to the public Internet. Repetitive requests for the same static assets and repeated external DNS resolutions exhaust the capacity of that access link.

Queuing theory proof. Traffic intensity is defined below, where L is the object length in bits, a is the average arrival rate of requests (objects per second), and R is the access-link capacity (bits per second).

$$I = (L \cdot a) / R$$

Traffic intensity (dimensionless)

As I approaches 1, the average queuing delay grows without bound:

$$I \to 1 \implies d_{queue} \to \infty$$

$$d_{total} = d_{proc} + d_{queue} + d_{trans} + d_{prop}, \quad d_{trans} = L / R$$

Queuing delay diverges as the access link saturates; total nodal delay model

EdgeShield serves repeated traffic at the edge. For cached assets, the WAN transmission delay ($d_{trans} = L / R$) and propagation delay (d_{prop}) across the access link drop to 0, and the load on the link, and therefore I, falls with every hit.

2. Architecture & Division of Work

Dimension	Partner A (MacBook M4, macOS)	Partner B (Linux Machine)
Assigned Subsystem	Web Caching Proxy Core	In-Memory DNS Resolver & Profiler
Transport Layer	TCP (SOCK_STREAM) | Port 8888	UDP (SOCK_DGRAM) | Port 5300
Core Mechanics	Welcoming & Connection Sockets, LRU Eviction	12-Byte Header Framing, Type A RR Packing
Validation Logic	Conditional GET (If-Modified-Since / 304)	In-Memory Zone, TTL Countdown, WAN Forward
Security / Policy	Disk Cache Quota (50 MB Cap)	DNS Sinkholing (0.0.0.0 for Trackers)
Primary Codefile	edgeshield_proxy.py	dns_resolver.py

3. Partner A: Completed Implementation & Empirical Results

Architecture built. Partner A implemented a multithreaded TCP proxy server in Python. It provides a Welcoming Socket (`listen(20)`) that dispatches one worker thread per client, parses raw HTTP/1.1 ASCII request headers, maintains local disk storage (`./edge_storage`), enforces LRU cache eviction at 50 MB, and applies a 30-second freshness TTL together with `If-Modified-Since` Conditional GET validation (HTTP 304 Not Modified).

Verified empirical benchmark results. The automated harness (`benchmark.py`) executed 10 sequential HTTP transactions against the proxy:

Iteration	Observed Latency	Transaction Payload	Operational State
Request 01	114.75 ms	931 bytes	CACHE MISS (Cold WAN Fetch)
Request 02	1.31 ms	931 bytes	CACHE HIT (Local Edge Delivery)
Request 03	0.56 ms	931 bytes	CACHE HIT (Local Edge Delivery)
Requests 04 - 10	0.49 ms - 0.86 ms (avg 0.71 ms)	931 bytes	CACHE HIT (Local Edge Delivery)
Performance Gain	**162.06x Speedup**	**0 bytes on access link**	**Queue congestion mitigated**

4. Cross-Platform Interconnection Setup (macOS + Linux)

Because the proxy runs on Partner A's MacBook and the DNS resolver runs on Partner B's Linux laptop, both machines must be on the same Local Area Network (Wi-Fi or mobile hotspot) to communicate.

Step A: Retrieve LAN IP Addresses

```
On macOS (Partner A): ipconfig getifaddr en0 -> e.g., 192.168.1.35 (MAC_IP)

On Linux (Partner B): hostname -I | awk '{print $1}' -> e.g., 192.168.1.42 (LINUX_IP)
```

Step B: Sockets Configuration

- [ ] **Partner A's proxy:** set `PROXY_HOST = '0.0.0.0'` and `PROXY_PORT = 8888` so the proxy accepts packets from the LAN.
- [ ] **Partner B's resolver:** map `edgeshield.lan` to `MAC_IP` in the `LOCAL_ZONES` dictionary and bind UDP to `0.0.0.0:5300`.

5. Partner B: Step-by-Step To-Do List & Implementation Checklist

Partner B must complete the following checklist on Linux to finish the DNS subsystem.

Task 1: Verify Linux Socket Environment & Firewall

- [ ] Verify Python 3: `python3 --version`
- [ ] Allow UDP 5300 through the firewall: `sudo ufw allow 5300/udp`
- [ ] Ensure port 5300 is free: `sudo lsof -i :5300`

Task 2: Implement dns_resolver.py Logic

- [ ] **UDP datagram socket:** bind `socket(AF_INET, SOCK_DGRAM)` to `('0.0.0.0', 5300)`.
- [ ] **12-byte header parsing:** unpack with `struct.unpack('!HHHHHH', buf[:12])` (ID, Flags, QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT).
- [ ] **Type A RR encoding:** resolve `edgeshield.lan` by packing a Type A record that points to Partner A's Mac IP.
- [ ] **Ad/tracker sinkholing:** return `0.0.0.0` immediately if the domain matches `doubleclick.net` or another tracker list entry.
- [ ] **Recursive WAN fallback:** forward cache misses to upstream DNS (`1.1.1.1:53`) over UDP and relay the response to the client.

Task 3: Run Validation Tests on Linux

- [ ] **Local zone:** `dig @127.0.0.1 -p 5300 edgeshield.lan A` -> status NOERROR, IP = Partner A's Mac IP, query time 0 ms.
- [ ] **Sinkhole:** `dig @127.0.0.1 -p 5300 doubleclick.net A` -> status NOERROR, IP = 0.0.0.0.
- [ ] **WAN fallback:** `dig @127.0.0.1 -p 5300 google.com A` -> resolves a public IP via the Cloudflare upstream.

Task 4: Wireshark Verification

- [ ] Run Wireshark on Linux, capturing the loopback (`lo`) or Wi-Fi interface.
- [ ] Filter with `udp.port == 5300` and capture the DNS query and answer frames, confirming response header flags `0x8180`.

6. Joint Live Demonstration Script for Faculty Evaluation

Step	Action & Command	Observed Outcome	Key Theoretical Concept
1. Intro	Explain the access-link bottleneck	Frames project goals	Traffic intensity I = La / R
2. DNS Test	Mac: `dig @<LINUX_IP> -p 5300 edgeshield.lan`	Linux returns the Mac IP in 0 ms	UDP SOCK_DGRAM, local directory
3. Cache Miss	Linux: `curl -x http://<MAC_IP>:8888 http://example.com/`	Status 200 OK, full payload, ~100 ms	TCP welcoming socket, WAN RTT delay
4. Cache Hit	Linux: repeat the identical curl command	Status 304 / pure hit in <1 ms	Conditional GET, d_{trans} = 0 across WAN
5. Benchmark	Mac: `python3 benchmark.py`	Prints 162.06x speedup	Queue bloat eliminated at the edge